

Problem 1: Histogram Equalization

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image in grayscale
image = cv2.imread('flower.png', cv2.IMREAD_GRAYSCALE)

# Utility function to display images
def display_images(titles, images):
    fig, axs = plt.subplots(1, len(images), figsize=(15, 5))
    for ax, title, img in zip(axs, titles, images):
        ax.imshow(img, cmap='gray')
        ax.set_title(title)
        ax.axis('off')
    plt.show()

# 1. Global Histogram Equalization
def global_histogram_equalize(image):
    hist, bins = np.histogram(image.flatten(), 256, [0, 256])
    cdf = hist.cumsum()
    cdf_normalized = cdf * hist.max() / cdf.max()

    cdf_m = np.ma.masked_equal(cdf, 0)
    cdf_m = (cdf_m - cdf_m.min()) * 255 / (cdf_m.max() - cdf_m.min())
    cdf = np.ma.filled(cdf_m, 0).astype('uint8')

    return cdf[image]

# 2. Local Histogram Equalization
def local_histogram_equalize(image, block_size):
    # Split image into blocks
    h, w = image.shape
    equalized_image = np.zeros_like(image)

    for y in range(0, h, block_size):
        for x in range(0, w, block_size):
            block = image[y:y+block_size, x:x+block_size]
            equalized_block = global_histogram_equalize(block)
            equalized_image[y:y+block_size, x:x+block_size] = equalized_block

    return equalized_image

# 3. CLAHE (using OpenCV)
def apply_clahe(image):
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
```

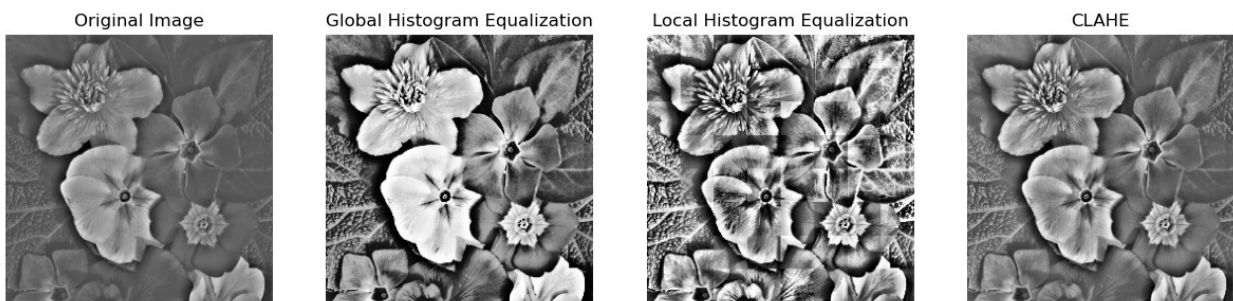
```

    return clahe.apply(image)

# Plot and compare the results
global_eq_img = global_histogram_equalize(image)
local_eq_img = local_histogram_equalize(image, 32) # Using a 32x32
block size for example
clahe_img = apply_clahe(image)

display_images(
    ["Original Image", "Global Histogram Equalization", "Local
Histogram Equalization", "CLAHE"],
    [image, global_eq_img, local_eq_img, clahe_img]
)

```



Problem 2: Intensity Transformation - Gamma Correction

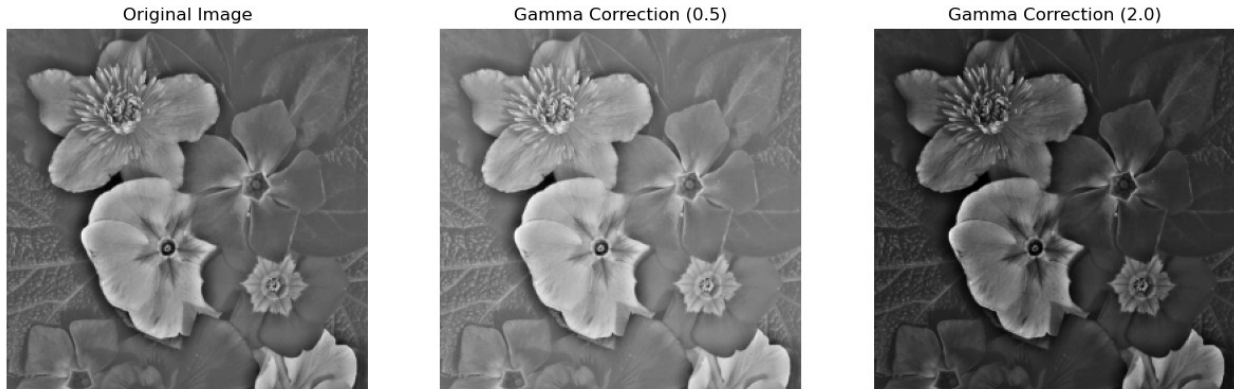
```

def gamma_correction(image, gamma):
    # Normalize the image to range [0, 1]
    normalized_img = image / 255.0
    gamma_corrected_img = np.power(normalized_img, gamma)
    # Scale back to [0, 255]
    return np.uint8(gamma_corrected_img * 255)

# Example gamma correction
gamma_0_5 = gamma_correction(image, 0.5) # Brightening
gamma_2 = gamma_correction(image, 2.0)   # Darkening

display_images(
    ["Original Image", "Gamma Correction (0.5)", "Gamma Correction
(2.0)"],
    [image, gamma_0_5, gamma_2]
)

```



Problem 3: Median Filtering with Salt-and-Pepper Noise

```
# Adding Salt-and-Pepper Noise
def add_salt_pepper_noise(image, prob):
    noisy_image = np.copy(image)
    h, w = image.shape
    # Salt
    num_salt = np.ceil(prob * h * w / 2)
    coords = [np.random.randint(0, i - 1, int(num_salt)) for i in
image.shape]
    noisy_image[coords[0], coords[1]] = 255

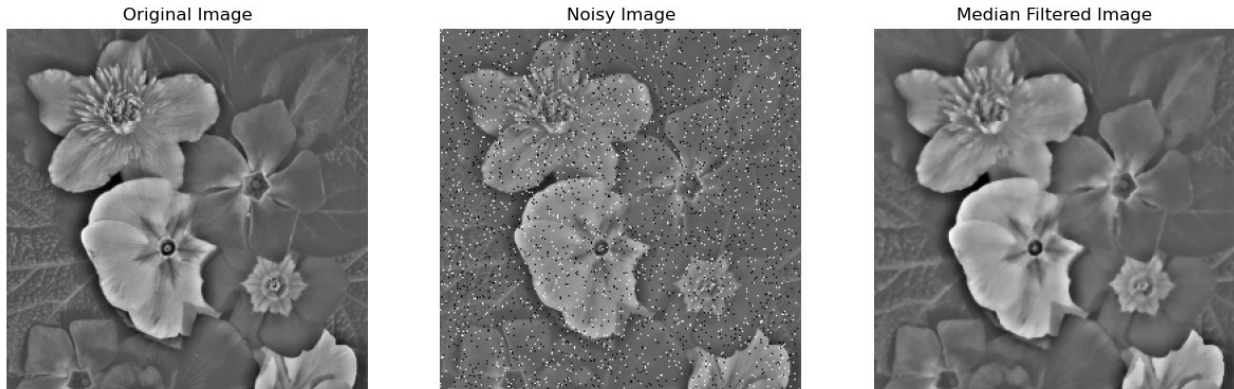
    # Pepper
    num_pepper = np.ceil(prob * h * w / 2)
    coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in
image.shape]
    noisy_image[coords[0], coords[1]] = 0

    return noisy_image

# Median Filtering
def median_filter(image, kernel_size):
    return cv2.medianBlur(image, kernel_size)

# Example with Salt-and-Pepper noise and Median Filtering
noisy_image = add_salt_pepper_noise(image, 0.05)
filtered_image = median_filter(noisy_image, 3)

display_images(
    ["Original Image", "Noisy Image", "Median Filtered Image"],
    [image, noisy_image, filtered_image]
)
```



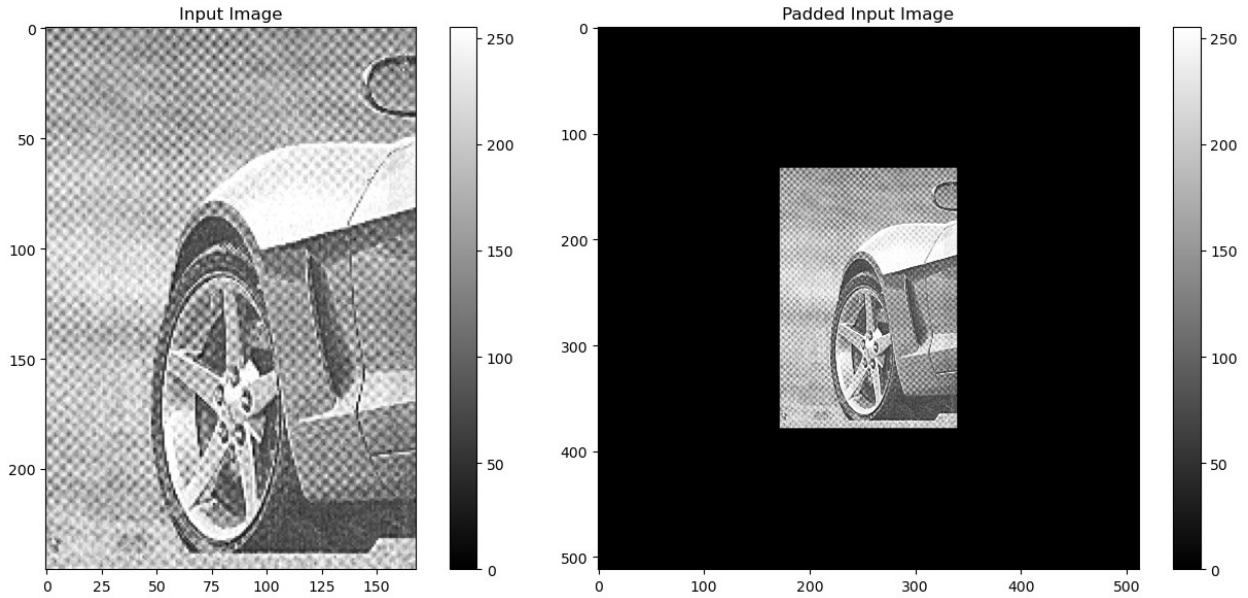
Problem 4: Butterworth Notch Reject Filtering in Frequency Domain

(i) car.tif

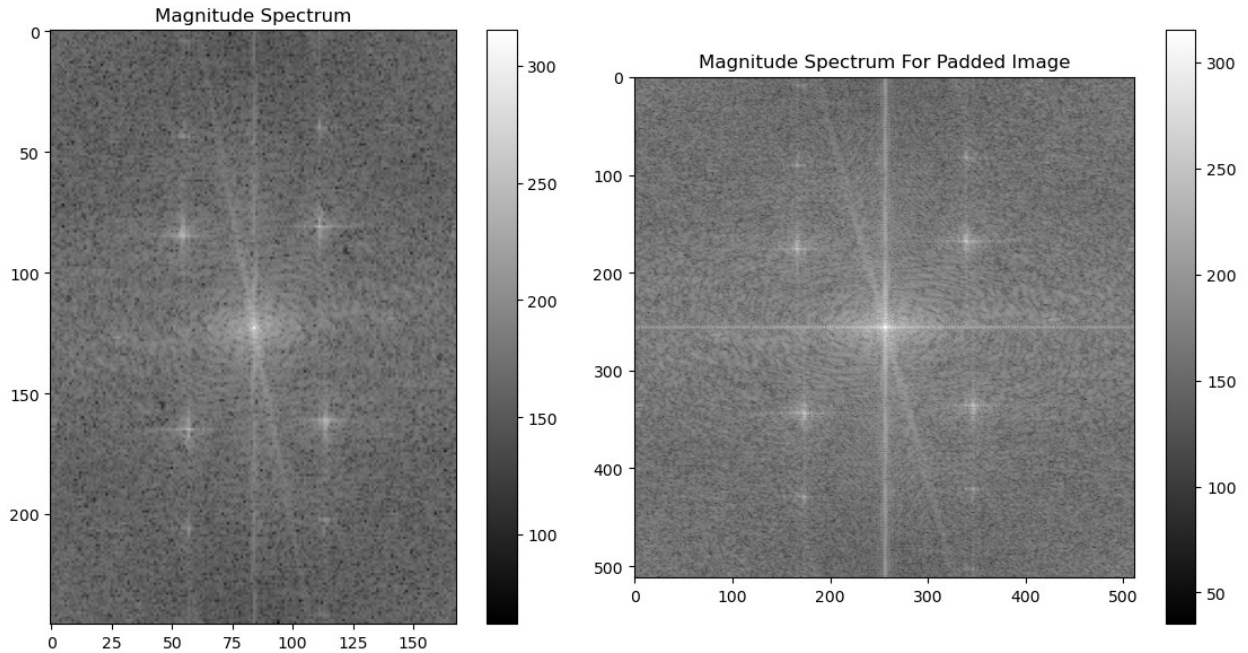
Parameters: $n = 15$; $D_0 = 20$; $u_1 = 84$; $v_1 = -176$; $u_2 = 83$; $v_2 = -86$; $u_3 = 90$; $v_3 = 74$; $u_4 = 94$; $v_4 = 164$.

```
import math
def addingPad(im, xPad, yPad):
    new_im = im.copy()
    new_x = math.floor((xPad - new_im.shape[0])/2)
    new_y = math.floor((yPad - new_im.shape[1])/2)
    new_im = np.pad(new_im, ((new_x, new_x), (new_y, new_y)),
mode='constant',constant_values=0)
    return new_im

original_im = cv2.imread('Car.tif',cv2.IMREAD_GRAYSCALE)
im_pad = addingPad(original_im, 512, 512)
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
ax = axes.ravel()
a0 = ax[0].imshow(original_im, cmap='gray')
ax[0].set_title("Input Image")
fig.colorbar(a0, ax=ax[0])
a1 = ax[1].imshow(im_pad, cmap="gray")
ax[1].set_title("Padded Input Image")
fig.colorbar(a1, ax=ax[1])
fig.tight_layout()
plt.show()
```



```
f = np.fft.fft2(original_im)
fshift = np.fft.fftshift(f)
log_mag = 20*np.log(np.abs(fshift))
# log_mag = 20*np.log(np.abs(fshift))
f_pad = np.fft.fft2(im_pad)
fshift_pad = np.fft.fftshift(f_pad)
log_mag_pad = 20*np.log(np.abs(fshift_pad))
# log_mag_pad = 20*np.log(np.abs(fshift_pad))
fig, axes = plt.subplots(1, 2, figsize=(12, 6))
ax = axes.ravel()
a0 = ax[0].imshow(log_mag, cmap="gray")
ax[0].set_title("Magnitude Spectrum")
fig.colorbar(a0, ax=ax[0])
a1 = ax[1].imshow(log_mag_pad, cmap="gray")
ax[1].set_title("Magnitude Spectrum For Padded Image")
fig.colorbar(a1, ax=ax[1])
fig.tight_layout()
plt.show()
```



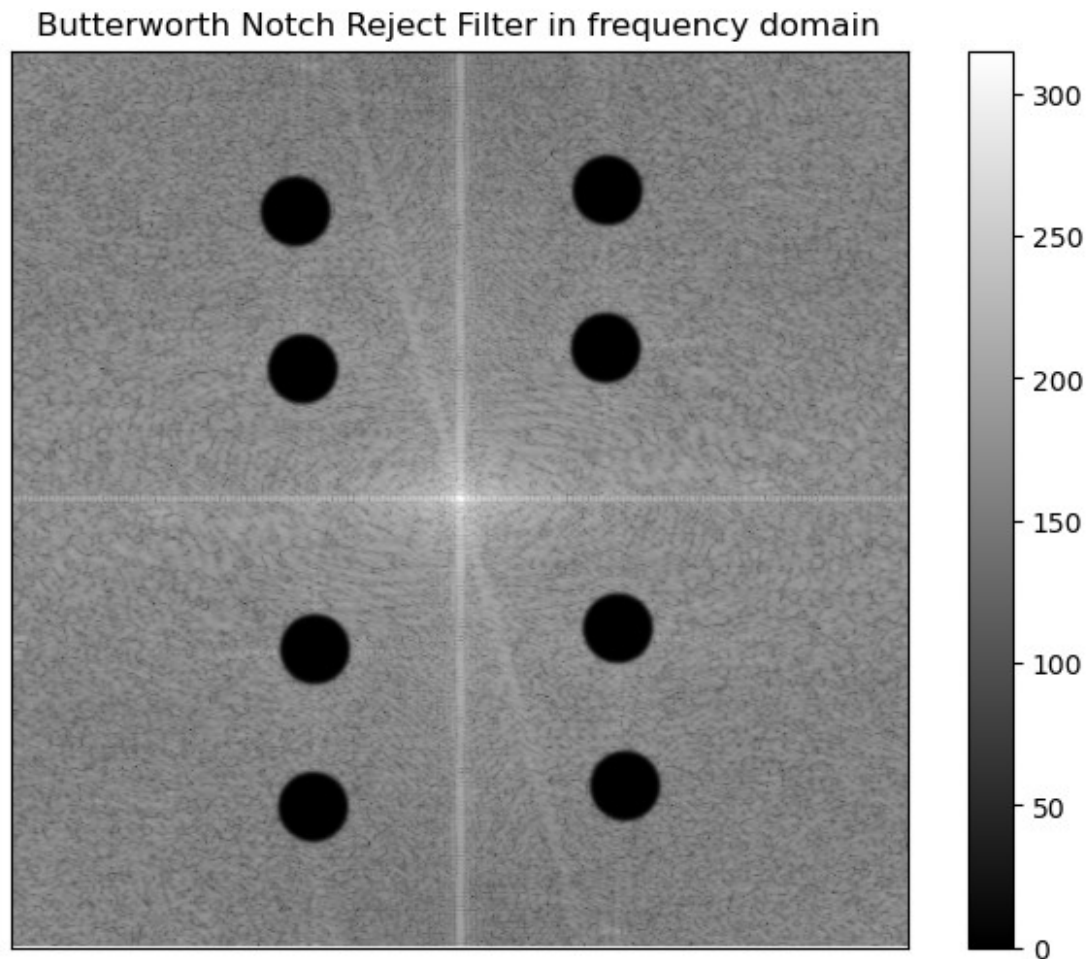
```
def hnr(im):
    K = 4
    D0 = 20
    n = 15
    u1 = 84
    v1 = -176
    u2 = 83
    v2 = -86
    u3 = 90
    v3 = 74
    u4 = 94
    v4 = 164
    u_i = [u1,u2,u3,u4]
    v_i = [v1,v2,v3,v4]
    x_axis = np.linspace(-256,255,512)
    y_axis = np.linspace(-256,255,512)
    [u,v] = np.meshgrid(x_axis, y_axis)
    new_im = np.zeros((im.shape[0],im.shape[1]))
    for i in range(im.shape[0]):
        for j in range(im.shape[1]):
            pixel = 1
            for k in range(K):
                D_k = ((u[i,j] - u_i[k])**2 + (v[i,j] -
v_i[k])**2)**0.5
                D_k_minus = ((u[i,j] + u_i[k])**2 + (v[i,j] +
v_i[k])**2)**0.5
                pixel *= (1/(1 + (D0/D_k)**(2*n)))*(1/(1 +
(D0/D_k_minus)**(2*n)))
            new_im[i,j] = pixel
```

```
return new_im
```

```
H_NR = hnr(im_pad)
NR_mag_freq = log_mag_pad*H_NR
plt.figure(figsize=(8, 6))
plt.imshow(NR_mag_freq, cmap = 'gray')
plt.xticks([]), plt.yticks([])
cbar = plt.colorbar()
plt.title('Butterworth Notch Reject Filter in frequency domain')
plt.show()
```

/tmp/ipykernel_13495/3203649151.py:25: RuntimeWarning: divide by zero encountered in double_scalars

```
pixel *= (1/(1 + (D0/D_k)**(2*n)))*(1/(1 + (D0/D_k_minus)**(2*n)))
```



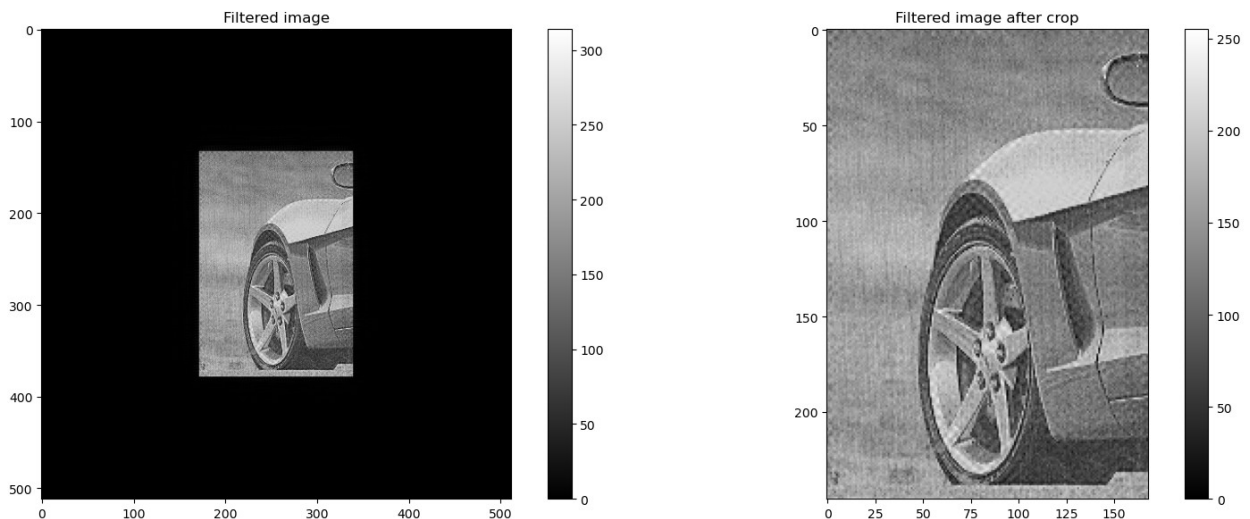
```
def cropImg(origin, im):
    x = math.floor((512 - origin.shape[0])/2)
    y = math.floor((512 - origin.shape[1])/2)
    new_im = im[x:(x+origin.shape[0]),y:(y+origin.shape[1])]
```

```

return new_im

sptr_NR = fshift_pad*H_NR
NR_image = abs(np.fft.ifft2(sptr_NR))
cropped_image = cropImg(original_im, NR_image)
normalized_cropped_image = np.zeros((cropped_image.shape[0],
cropped_image.shape[1]))
cv2.normalize(cropped_image, normalized_cropped_image, 0, 255,
cv2.NORM_MINMAX)
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
ax = axes.ravel()
a0 = ax[0].imshow(NR_image, cmap="gray")
ax[0].set_title("Filtered image")
fig.colorbar(a0, ax=ax[0])
a1 = ax[1].imshow(normalized_cropped_image, cmap="gray")
ax[1].set_title("Filtered image after crop")
fig.colorbar(a1, ax=ax[1])
fig.tight_layout()
plt.show()

```



(ii) street.png.

Parameters: $K = 2$; $D0 = 20$; $n = 10$; $u1 = 0$; $v1 = 165$; $u2 = 165$; $v2 = 0$.

```

def bnrff(im):
    original_im = im.copy()
    im_pad = addingPad(original_im, 512, 512)
    f = np.fft.fft2(original_im)
    fshift = np.fft.fftshift(f)
    log_mag = 20*np.log(np.abs(fshift))
    f_pad = np.fft.fft2(im_pad)
    fshift_pad = np.fft.fftshift(f_pad)
    log_mag_pad = 20*np.log(np.abs(fshift_pad))

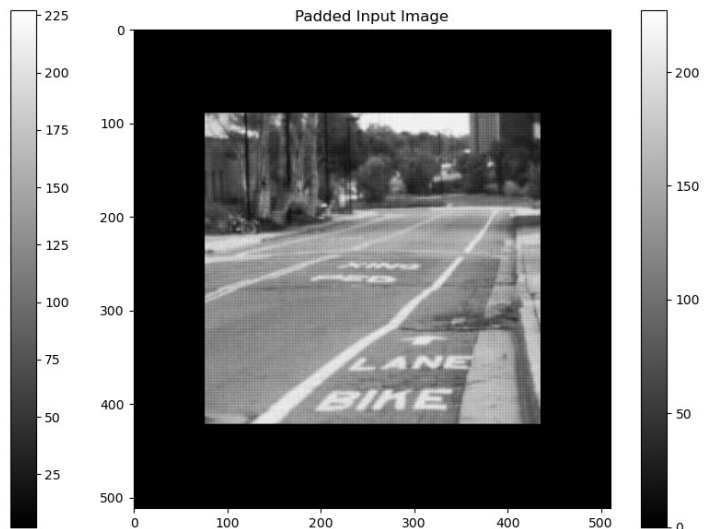
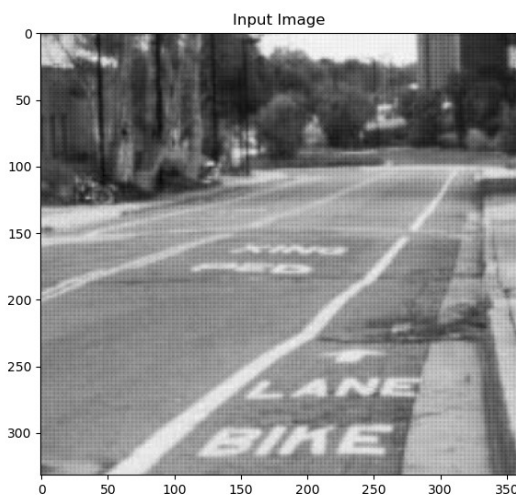
```

```

H_NR = hnr(im_pad)
NR_mag_freq = log_mag_pad*H_NR
sptr_NR = fshift_pad*H_NR
NR_image = abs(np.fft.ifft2(sptr_NR))
cropped_image = cropImg(original_im, NR_image)
return cropped_image

grayscale_im = cv2.imread('Street.png',cv2.IMREAD_GRAYSCALE)
im_pad = addingPad(grayscale_im, 512, 512)
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
ax = axes.ravel()
a0 = ax[0].imshow(grayscale_im, cmap='gray')
ax[0].set_title("Input Image")
fig.colorbar(a0, ax=ax[0])
a1 = ax[1].imshow(im_pad, cmap="gray")
ax[1].set_title("Padded Input Image")
fig.colorbar(a1, ax=ax[1])
fig.tight_layout()

```



```

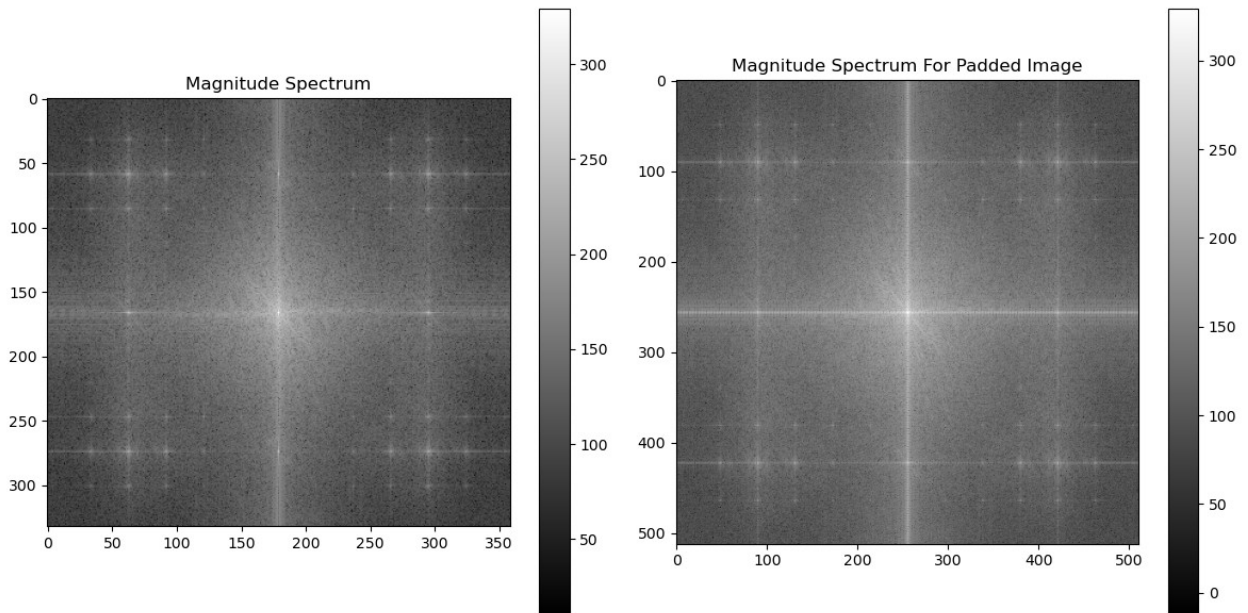
f = np.fft.fft2(grayscale_im)
fshift = np.fft.fftshift(f)
log_mag = 20*np.log(np.abs(fshift))
# log_mag = 20*np.(np.abs(fshift))
f_pad = np.fft.fft2(im_pad)
fshift_pad = np.fft.fftshift(f_pad)
log_mag_pad = 20*np.log(np.abs(fshift_pad))
# log_mag_pad = 20*np.(np.abs(fshift_pad))
fig, axes = plt.subplots(1, 2, figsize=(12, 6))
ax = axes.ravel()
a0 = ax[0].imshow(log_mag, cmap="gray")
ax[0].set_title("Magnitude Spectrum")
fig.colorbar(a0, ax=ax[0])

```

```

a1 = ax[1].imshow(log_mag_pad, cmap="gray")
ax[1].set_title("Magnitude Spectrum For Padded Image")
fig.colorbar(a1, ax=ax[1])
fig.tight_layout()
plt.show()

```



```

def hnr(im):
    K = 2
    D0 = 20
    n = 10
    u1 = 0
    v1 = 165
    u2 = 165
    v2 = 0
    u_i = [u1,u2]
    v_i = [v1,v2]
    x_axis = np.linspace(-256,255,512)
    y_axis = np.linspace(-256,255,512)
    [u,v] = np.meshgrid(x_axis, y_axis)
    new_im = np.zeros((im.shape[0],im.shape[1]))
    for i in range(im.shape[0]):
        for j in range(im.shape[1]):
            pixel = 1
            for k in range(K):
                D_k = ((u[i,j] - u_i[k])**2 + (v[i,j] -
v_i[k])**2)**0.5
                D_k_minus = ((u[i,j] + u_i[k])**2 + (v[i,j] +
v_i[k])**2)**0.5
                pixel *= (1/(1 + (D0/D_k)**(2*n)))*(1/(1 +
(D0/D_k_minus)**(2*n)))

```

```

        new_im[i,j] = pixel
    return new_im

```

```

H_NR = hnr(im_pad)
NR_mag_freq = log_mag_pad*H_NR
plt.figure(figsize=(8, 6))
plt.imshow(NR_mag_freq, cmap = 'gray')
plt.xticks([]), plt.yticks([])
cbar = plt.colorbar()
plt.title('Butterworth Notch Reject Filter in frequency domain')
plt.show()

```

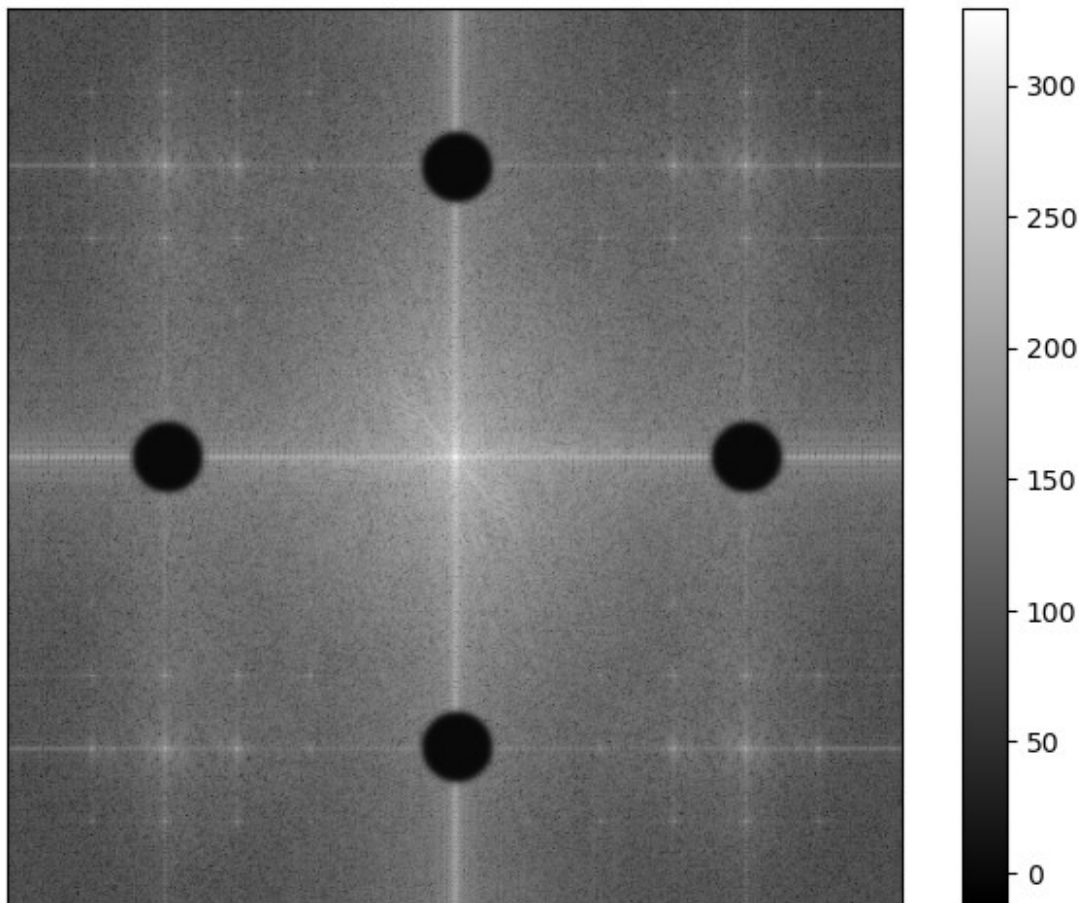
/tmp/ipykernel_13495/1566346747.py:21: RuntimeWarning: divide by zero encountered in double_scalars

```

    pixel *= (1/(1 + (D0/D_k)**(2*n)))*(1/(1 + (D0/D_k_minus)**(2*n)))

```

Butterworth Notch Reject Filter in frequency domain



```

sptr_NR = fshift_pad*H_NR
NR_image = abs(np.fft.ifft2(sptr_NR))
cropped_image = cropImg( grayscale_im, NR_image)
normalized_cropped_image = np.zeros((cropped_image.shape[0],

```

```

cropped_image.shape[1]))
cv2.normalize(cropped_image, normalized_cropped_image, 0, 255,
cv2.NORM_MINMAX)
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
ax = axes.ravel()
a0 = ax[0].imshow(NR_image, cmap="gray")
ax[0].set_title("Filtered image")
fig.colorbar(a0, ax=ax[0])
a1 = ax[1].imshow(normalized_cropped_image, cmap="gray")
ax[1].set_title("Filtered image after crop")
fig.colorbar(a1, ax=ax[1])
fig.tight_layout()
plt.show()

```

